

JAVA DEVELOPER INTERNSHIP

TASK 3

Collection & Exception Handling

Internship Organization

ALFIDO TECH

Submitted By : Shreya Verma

GitHub Repository:

https://github.com/vermashreya127-blip/AlifidoTech-Task3_CollectionAndExceptionHandling

Objective

The primary objective of this project is to understand and implement the Java Collections Framework and Exception Handling concepts in a practical application. The project demonstrates how different collection classes such as ArrayList, HashSet, and HashMap can be used to store, organize, and manage data efficiently.

Another objective is to learn how to handle runtime errors using try-catch blocks and create custom exceptions for validating user input. Through this project, student information is stored and managed while ensuring that invalid data is detected and handled properly.

This project also helps in improving programming skills related to data management, error handling, object-oriented programming concepts, and the effective use of Java's built-in collection classes. The overall goal is to develop a reliable and user-friendly student management system that demonstrates efficient data storage and robust exception handling techniques.

Tools and Technologies Used

- Tool| Purpose
- Java| Programming Language
- VS Code| Code Editor
- GitHub| Version Control & Repository Hosting

Problem Statement

Develop a Java application that demonstrates the use of:

- ArrayList
- HashSet
- HashMap
- Try-Catch blocks
- Custom Exception Handling

The program should store student information and validate student age using a custom exception.

Description

The Student Management System is a Java-based application developed to demonstrate the practical implementation of Java Collections Framework and Exception Handling concepts. The project focuses on storing, organizing, and managing student information efficiently while ensuring proper validation of user input.

In this application, the user is asked to enter student details such as name and age. The entered data is then stored using different collection classes available in Java. An ArrayList is used to store student names in a dynamic list, a HashSet is used to maintain unique student records

by preventing duplicate entries, and a HashMap is used to store student names along with their corresponding ages in key-value pair format.

The project also incorporates Exception Handling to improve the reliability and stability of the application. A custom exception named `InvalidAgeException` has been created to validate student age. If the user enters an age less than 18, the program throws a custom exception and displays an appropriate error message. This helps in preventing invalid data from being processed by the system.

Additionally, try-catch blocks are used to handle unexpected runtime errors and ensure smooth program execution. This makes the application more robust and user-friendly. The project provides hands-on experience in working with Java collections, custom exceptions, and error-handling mechanisms while strengthening problem-solving and programming skills.

Overall, this project serves as an excellent example of how data structures and exception handling techniques can be combined to create efficient, reliable, and maintainable Java applications.

Collections Used:

ArrayList

Used to store student names.

HashSet

Used to store unique student names and prevent duplicates.

HashMap

Used to store student names along with their ages.

Try-Catch

Used to handle runtime exceptions.

Custom Exception

InvalidAgeException is created to validate student age.

Algorithm

1. Start the program.
2. Create Scanner object for user input.
3. Create ArrayList, HashSet, and HashMap.
4. Ask the user to enter student name.
5. Ask the user to enter student age.
6. Check if age is less than 18.
7. If age is less than 18, throw InvalidAgeException.
8. Otherwise store the data in collections.
9. Display stored student information.
10. Handle errors using try-catch.
11. Close the scanner.
12. End the program.

Source Code

```
J StudentManager.java X  README.md  ▶  ▢  ...

J StudentManager.java > StudentManager

1  import java.util.*;
2
3  class InvalidAgeException extends Exception {
4      public InvalidAgeException(String message) {
5          super(message);
6      }
7  }
8
9  public class StudentManager {
10     public static void main(String[] args) {
11
12         Scanner sc = new Scanner(System.in);
13
14         ArrayList<String> students = new ArrayList<>();
15         HashSet<String> uniqueStudents = new HashSet<>();
16         HashMap<String, Integer> studentAge = new HashMap<>();
17
18         try {
19             System.out.print(s: "Enter Student Name: ");
20             String name = sc.nextLine();
21
22             System.out.print(s: "Enter Student Age: ");
23             int age = sc.nextInt();
24
25             if (age < 18) {
26                 throw new InvalidAgeException(message: "Age must be 18");
27             }
28
29             students.add(name);
```

J StudentManager.java X

① README.md



J StudentManager.java > StudentManager

```
9  public class StudentManager {
10     public static void main(String[] args) {
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29         students.add(name);
30         uniqueStudents.add(name);
31         studentAge.put(name, age);
32
33         System.out.println(x: "\n--- Student Details ---");
34         System.out.println("ArrayList: " + students);
35         System.out.println("HashSet: " + uniqueStudents);
36         System.out.println("HashMap: " + studentAge);
37
38     } catch (InvalidAgeException e) {
39         System.out.println("Custom Exception: " + e.getMessage());
40
41     } catch (Exception e) {
42         System.out.println("Error: " + e.getMessage());
43
44     } finally {
45         sc.close();
46     }
47 }
48 }
```

Output - Valid Input

Input:

Name: Shreya

Age: 20

Output:

Student List: [Shreya]

Unique Students: [Shreya]

Student Age Map: {Shreya=20}

```
Enter Student Name: Shreya
```

```
Enter Student Age: 20
```

```
--- Student Details ---
```

```
ArrayList: [Shreya]
```

```
HashSet: [Shreya]
```

```
HashMap: {Shreya=20}
```

Output - Exception Handling

Input:

Name: Shreya

Age: 15

Output:

Error: Age must be 18 or ab

```
Enter Student Name: Shreya
```

```
Enter Student Age: 15
```

```
Custom Exception: Age must be 18 or above.
```

Conclusion

This project successfully demonstrates the implementation of Java Collections Framework and Exception Handling concepts. ArrayList, HashSet, and HashMap were used for efficient data management, while try-catch blocks and a custom exception ensured proper error handling and input validation. The project improved understanding of Java collections and exception management techniques.

